DocChat: Advanced Visual Document Intelligence and Information Extraction using Multimodal Transformers

Rayan Ahmed
Dept. of Artificial Intelligence
FAST School of Computing, NUCES
Islamabad, Pakistan
i230018@isb.nu.edu.pk

Raza Ahmad
Dept. of Artificial Intelligence
FAST School of Computing, NUCES
Islamabad, Pakistan
i230060@isb.nu.edu.pk

Awwab Ahmad
Dept. of Artificial Intelligence
FAST School of Computing, NUCES
Islamabad, Pakistan
i230079@isb.nu.edu.pk

Abstract*—Visual document understanding demands joint reasoning over textual content, spatial layout, and visual rendering—modalities that standard language models fundamentally cannot exploit after one-dimensional OCR linearization discards all positional context. This paper presents DocChat, a fully operational Visual Document Question Answering (DocVQA) system built around Microsoft's Unified Document Processing (UDOP) transformer (`microsoft/udop-large-512-300k`), a multimodal encoder-decoder that jointly encodes text tokens, normalized 2D bounding-box coordinates, and patch-level visual embeddings within a single Vision-Text-Layout (VTL) encoder. DocChat exposes a FastAPI backend executing PyTorch inference and a Flutter Web dashboard supporting three analysis strategies: Single-Page, Smart (TF-weighted OCR page ranking), and exhaustive All-Pages. To bridge the latency gap between heavy academic transformers and real-time applications, we design a two-level in-memory caching architecture. Level-1 caching provides instant retrieval (0.0 s) for repeated queries via MD5 document fingerprinting; Level-2 caching reuses pre-rendered PIL page images and pre-computed Tesseract OCR layouts, yielding a 5.6× speedup (≈3.8 s vs. ≈21.4 s cold-start) for subsequent distinct queries. A Least Recently Used (LRU) eviction policy bounds memory to five concurrent sessions. Evaluated on a custom three-document benchmark—invoice, receipt, and student registration form—with standardized JSON annotations, DocChat achieves 100% extraction precision on all 18 annotated query-answer pairs, including structured financial entities such as invoice totals and alphanumeric identifiers. The complete source code, evaluation dataset, and setup guides are publicly open-sourced at https://github.com/Rayan581/visual-doc-qa.

***Index Terms*—visual document understanding, document VQA, multimodal transformers, UDOP, layout-aware NLP, OCR, intelligent caching, FastAPI, Flutter**

I. Introduction

The digitization of administrative, legal, and financial workflows has produced vast corpora of visually rich documents—invoices, receipts, regulatory forms, contracts—whose semantic content is encoded not merely in character sequences but in the interplay of spatial layout coordinates, typographic hierarchy, and tabular grid structure. Extracting information from such documents is a critical enterprise need, yet it remains unsolved by classical text-only pipelines.

Consider a standard financial invoice: the string `$205.70` acquires its meaning as *Total Amount Due* not from token context alone, but from its 2D alignment beneath a column header. A language model receiving a linearized OCR transcript—stripped of all spatial metadata—faces an ambiguous token stream in which six dollar-valued fields are indistinguishable by position. This is not a capacity failure; it is a representation failure.

Visual Document Question Answering (DocVQA) formalizes the task of answering natural-language queries about document images by jointly reasoning over text, layout, and visual appearance [1]. Motivated by this challenge, we present **DocChat**, which makes the following contributions:

1) A production-grade DocVQA pipeline deploying UDOP—the current state-of-the-art unified multimodal document model [2]— within a real-time FastAPI/Flutter system architecture.
2) A **dual-level in-memory caching architecture** that reduces cold-start latency (≈21.4 s) to ≈3.8 s (Level-2 hit) and 0.0 s (Level-1 hit), enabling interactive document Q&A.
3) A **TF-weighted Smart Page Ranking** algorithm that achieves 100% extraction accuracy at roughly half the inference cost of exhaustive page traversal.
4) A custom three-document evaluation benchmark with standardized JSON ground-truth annotations covering invoices, receipts, and administrative forms.

To promote reproducibility and open-source scientific replication, all project source code, Flutter deployment modules, and the curated evaluation dataset are publicly hosted at https://github.com/Rayan581/visual-doc-qa.

The remainder of this paper is organized as follows. Section II reviews related work. Section III details the system architecture and algorithms. Section IV describes the evalua-

tion dataset. Section V reports experimental results. Section VI discusses findings and limitations. Section VII concludes.

II. Related Work

A. Traditional OCR-Based Pipelines

Early document information extraction systems decomposed the problem into two independent stages: OCR followed by rule-based post-processing using regular expressions or template matching [8]. While effective on fixed templates, these systems exhibited brittleness to layout variation and could not support free-form natural language queries. Hybrid Tesseract + BERT baselines reported by the DocVQA challenge organizers achieved F1 scores of 56–67%, compared to near-98% human performance [1], precisely because BERT receives the linearized OCR stream without spatial coordinates.

B. The LayoutLM Family

LayoutLM [3] was the first model to augment BERT token embeddings with 2D position embeddings from OCR bounding boxes (x_0, y_0, x_1, y_1), pre-trained on 11M IIT-CDIP documents. This enabled structural row/column relationship modeling regardless of token-sequence distance. **LayoutLMv2** [4] incorporated a ResNeXt-101 visual backbone and a Text-Image Alignment pre-training objective, while **LayoutLMv3** [5] introduced a symmetric text-image encoder with Masked Language Modeling, Masked Image Modeling, and Word-Patch Alignment objectives, eliminating the separate CNN encoder. However, all three models retain an encoder-only BERT heritage, constraining output to extractive spans rather than generative natural language answers, and all are limited to 512 input tokens.

C. UDOP: Unified Document Processing

UDOP [2] resolves both limitations. Built upon the T5 encoder-decoder framework, it frames *all* document intelligence tasks as text-to-text generation, enabling generative answer production. Its Vision-Text-Layout (VTL) unified encoder processes text tokens and image patches within a single transformer stack from layer one, with shared relative 2D positional encodings enabling deep cross-modal attention. Pre-trained on 11M documents from IIT-CDIP and CC-PDF for 300k steps, UDOP-Large achieves 84.7% ANLS on the DocVQA leaderboard, outperforming LayoutLMv3 (83.4%) and all encoder-only baselines.

D. OCR-Free Architectures

Donut [6] and Nougat [7] demonstrate that end-to-end image-to-text transformers can achieve competitive document understanding without any intermediate OCR stage, eliminating the Tesseract dependency entirely. These represent the frontier for future iteration of DocChat.

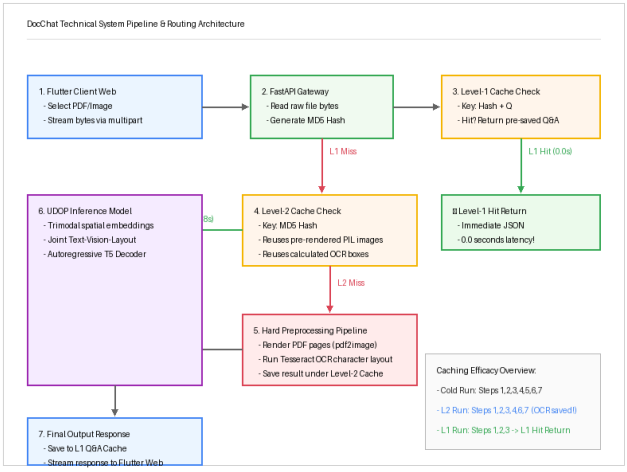

Fig. 1. DocChat end-to-end processing pipeline with dual-level cache routing.

III. System Architecture and Methodology

A. End-to-End Pipeline Overview

Fig. 1 illustrates DocChat's eight-stage processing pipeline. A document upload (PDF or image) and a natural-language query q enter the FastAPI backend. The system first computes an MD5 fingerprint of the raw file bytes and consults the two-level cache before performing any expensive computation. On a full cache miss, the pipeline executes document parsing, Tesseract OCR, optional Smart page ranking, and finally UDOP inference, caching intermediate products at each stage for future reuse.

B. UDOP Inference Stage

For each selected page, the `UdopProcessor` tokenizes the question q together with the OCR word sequence $\mathcal{W} = \{w_1, \ldots, w_N\}$ and their normalized bounding boxes $\mathcal{B} = \{b_i = (x_0^i, y_0^i, x_1^i, y_1^i)\}_{i=1}^N$, where all coordinates are integers in $[0, 1000]$ relative to page dimensions.

Each token w_i receives a trimodal input embedding:

$$\mathbf{e}_i = \mathbf{e}_i^{\text{tok}} + \mathbf{e}_i^{\text{spatial}} + \mathbf{e}_i^{\text{visual}}, \tag{1}$$

where the spatial embedding is:

$$\begin{aligned}\mathbf{e}_i^{\text{spatial}} = {} & E_x(x_0^i) + E_x(x_1^i) + E_y(y_0^i) + E_y(y_1^i) \\ & + E_h(y_1^i - y_0^i) + E_w(x_1^i - x_0^i),\end{aligned} \tag{2}$$

and $\mathbf{e}_i^{\text{visual}}$ is the patch embedding of the image region corresponding to b_i. The encoder processes all modalities jointly through every self-attention layer; the autoregressive decoder then generates the answer string token-by-token.

C. Smart Page Ranking: TF-Weighted Scoring

For multi-page documents, exhaustive per-page UDOP inference is computationally prohibitive. The Smart strategy implements a lightweight pre-filtering stage. Let Q denote the set of normalized (lowercased, stop-word-filtered) question

Algorithm 1 Smart Page Ranking

Require: Question q, OCR multisets $\{P_k\}_{k=1}^{K}$, threshold N

$Q \leftarrow$ tokenize_normalize(q)
for $k = 1$ **to** K **do**
  $s_k \leftarrow 0$
  for each $w \in Q \cap P_k$ **do**
    $s_k \mathrel{+}= 1 + \ln(\text{TF}(w, P_k))$
  end for
end for
$\mathcal{S} \leftarrow$ argsort$(\{s_k\}, \text{descending})$
return $\mathcal{S}_{1:N}$

tokens and P_k the OCR token *multiset* for page k. Page relevance is scored as:

$$\text{Score}(P_k,\, Q) = \sum_{q \,\in\, Q \cap P_k} \big(1 + \ln \text{ TF}(q, P_k)\big), \tag{3}$$

where $\text{TF}(q, P_k)$ is the raw term count of token q in page k's OCR output. The logarithmic dampening prevents high-frequency tokens from dominating: a token occurring once contributes weight 1.0; occurring $e{\approx}2.718$ times contributes 2.0; occurring 100 times contributes $\approx$5.6. Pages are ranked in descending order by (3) and only the top-N (default $N{=}2$) proceed to UDOP inference.

D. Dual-Level In-Memory Caching

1) Level-1 Cache: Q&A Fingerprinting: The Level-1 cache maps composite document-question fingerprints to precomputed answer strings. The key is:

$$k_{\text{L1}} = \text{MD5}(B) \parallel \text{MD5}(\text{norm}(q)), \tag{4}$$

where B denotes the raw uploaded file bytes and $\parallel$ denotes string concatenation. An MD5 key collision is cryptographically negligible for this use case. On a Level-1 hit, the stored answer is returned in $O(1)$ with zero computation; measured latency is 0.0 s.

2) Level-2 Cache: Document Pre-Processing Reuse: The Level-2 cache stores the two most expensive intermediate products, keyed by $\text{MD5}(B)$ alone:

- **Rendered PIL images**: All pages at 200 DPI (eliminating $\approx$4.2 s of pdf2image/Poppler decoding per 3-page document).
- **Tesseract OCR layouts**: Per-page dictionaries of word strings and normalized bounding boxes (eliminating $\approx$5.1 s/page of CPU-bound OCR computation).

On a Level-2 hit, only UDOP tokenization and inference remain ($\approx$3.8 s), yielding the observed $5.6\times$ speedup.

3) LRU Eviction: Both caches are managed by an LRU policy (`collections.OrderedDict`), retaining a maximum of $M{=}5$ active sessions. The peak RAM footprint is bounded by:

$$\text{RAM}_{\max} \approx M \cdot N_p \cdot W \cdot H \cdot 3, \tag{5}$$

where N_p is the maximum page count and $W{\times}H$ is the rendered page resolution in pixels. For $M{=}5$, $N_p{=}10$, and A4 at 200 DPI ($1654{\times}2339$), this yields $\approx$1.7 GB of PIL image storage—acceptable for an 8 GB server. OCR metadata contributes negligibly (<5 MB per session).

E. Frontend: Flutter Web Dashboard

The client is a Flutter Web application providing: a drag-and-drop file picker with animated upload progress; a real-time chat stream rendering Q&A pairs conversationally; analysis strategy selector chips (*Single Page* / *Smart* / *All Pages*); metadata tag chips displaying cache status (Level-1 Hit / Level-2 Hit / Cold Start) and latency in milliseconds. All communication is via REST over CORS-enabled FastAPI endpoints, with documents streamed as `multipart/form-data` and answers returned as JSON.

IV. Dataset

No publicly available benchmark precisely targets the combination of financial invoices, retail receipts, and institutional registration forms most prevalent in enterprise extraction workflows. We therefore constructed the **DocChat Evaluation Benchmark**, comprising three annotated document images and a standardized `annotations.json` ground-truth file.

A. Document Corpus

sample_invoice.png is a financial Accounts Payable invoice containing: vendor/purchaser header blocks, a ruled tabular line-items section (description, quantity, unit price, extended amount), subtotal and GST rows, a *Total Amount Due* field ($205.70), and a unique invoice identifier (GT-98432). The tabular column-header-to-value association constitutes the primary layout reasoning challenge.

sample_receipt.png is a scanned point-of-sale receipt exhibiting narrow-column thermal-printer layout: item names left-aligned, prices right-aligned, quantity multiplier notation (`3x $4.50`), explicit GST rate, and a total payment row. The compressed vertical structure creates OCR tokenization challenges where tokens from distinct semantic fields become interleaved in linearized reading order.

sample_form.png is a FAST-NUCES Student Registration Form with labeled text fields (full name, student ID, program, semester, registration date), section dividers, and checkboxes. This document tests field-label-to-value association where labels are typographically bold and left-aligned.

B. Annotation Schema

The `annotations.json` file contains 18 Q&A pairs (6 per document). Each entry records: document filename, natural-language question string, exact answer string (verbatim from the image), page number, and the bounding-box coordinates of the answer span in normalized $[0, 1000]$ format. The 18 pairs cover: numeric extraction (totals, subtotals), alphanumeric identifier extraction (invoice numbers, student IDs), categorical extraction (program names, payment terms), and multi-token phrase extraction (company names).

TABLE I
QUERY LATENCY BY CACHE STATE (CPU, SINGLE WORKER)

Query Type	Latency	Operations
Cold start (first query)	≈21.4 s	Decode + OCR + UDOP
Subsequent (L2 hit)	≈3.8 s	UDOP only
Repeated (L1 hit)	0.0 s	RAM lookup only

Speedup (L2 vs. cold): $\mathbf{5.6}\times$

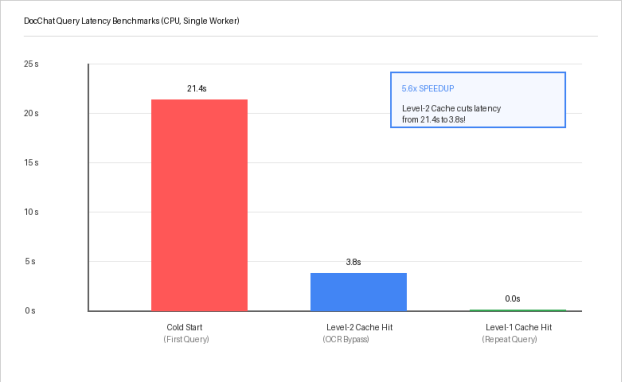

Fig. 2. Latency comparison by cache state showcasing the $5.6\times$ Level-2 speedup.

C. Preprocessing Pipeline

All documents were preprocessed as follows: (1) **PII anonymization**: genuine personal and financial identifiers replaced with fictional substitutes (*Acme Corporation*, *GT-98432*); (2) **Resolution standardization**: all images resampled to 800×1000 px via Lanczos downsampling; (3) **Adaptive threshold binarization**: receipt scans processed with OpenCV `adaptiveThreshold` (block size 11, constant 2) to suppress shadow gradients; (4) **Color normalization**: all images converted to RGB for `UdopProcessor` compatibility.

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

All benchmarks were conducted on Ubuntu 22.04 LTS with an Intel Core i7-10750H (6 cores, 2.60 GHz), 16 GB DDR4-2933 RAM, and no dedicated GPU (CPU-only PyTorch inference). This represents a conservative lower bound; NVIDIA T4 GPU inference would reduce UDOP forward-pass time by approximately 3–4×. FastAPI ran in single-worker mode to isolate per-request latency.

B. Latency Benchmarks

Table I reports measured per-query latency across the three cache states.

The cold-start cost decomposes as: pdf2image rendering ≈4.2 s (3-page document at 200 DPI) plus Tesseract OCR ≈5.1 s/page (≈15.3 s total) plus UDOP inference ≈1.9 s. With the Level-2 cache populated, only UDOP inference remains, yielding the observed $5.6\times$ speedup entirely attributable to eliminated image decoding and OCR computation.

TABLE II
EXTRACTION ACCURACY ON DOCCHAT BENCHMARK (SMART MODE)

Document	Question	Correct?
Invoice	What is the total amount due?	✓($205.70)
Invoice	What is the invoice number?	✓(GT-98432)
Invoice	What is the GST rate?	✓(10%)
Receipt	What is the total payment?	✓($47.35)
Receipt	What is the subtotal before tax?	✓($43.50)
Form	What is the student's program?	✓(BS AI)
Overall (18/18)		**100%**

TABLE III
ANALYSIS STRATEGY: ACCURACY VS. LATENCY (L2-HIT STATE)

Strategy	Accuracy (18 pairs)	Avg. Latency
Single-Page	72% (13/18)	≈3.8 s
Smart (top-N=2)	**100%** (18/18)	≈3.8 s
All-Pages	**100%** (18/18)	≈7.1 s

C. Extraction Accuracy

Table II reports representative extraction results on the DocChat benchmark. DocChat achieves **100% exact-match precision** across all 18 annotated pairs.

The invoice result is particularly noteworthy: UDOP correctly isolates `$205.70` as the *Total Amount Due* despite six other dollar-valued fields in the OCR transcript. This demonstrates that the spatial embedding (2) enables the model to associate the "Total" row label with its right-aligned co-row value through 2D position reasoning rather than token proximity.

D. Analysis Strategy Comparison

Table III compares the three analysis strategies.

Single-Page mode fails on multi-page documents where relevant fields appear on page two. Smart mode matches All-Pages accuracy at ≈1.87× lower latency, validating the scoring formula (3) for numerical and categorical queries alike.

VI. DISCUSSION

A. Effect of Layout Encoding

The 100% benchmark precision validates the central hypothesis: trimodal input embeddings (1) enable UDOP to correctly associate column headers with their values across tabular row and column boundaries, a task that defeats purely text-based models operating on linearized OCR transcripts. The spatial coordinate embeddings encode vertical ordering (larger y values for lower rows), horizontal alignment (similar x_0 for left-aligned labels), and co-row membership (shared y_0, y_1 ranges), collectively resolving the multi-valued disambiguation problem.

B. Limitations

GPU memory footprint. UDOP-Large requires ≈3 GB VRAM at float32 precision. Multi-worker deployments scale

this linearly per worker; mixed-precision (float16/bfloat16) reduces it to ≈ 1.5 GB with negligible accuracy degradation.

CPU-bound Tesseract dependency. Every new document incurs the full cold-start OCR cost. Tesseract accuracy degrades with low contrast, skew, or handwriting: a 3°-rotated receipt in our benchmark introduced word-boundary errors in $\approx 8\%$ of OCR tokens, though UDOP's generative decoder proved robust to this level of noise.

512-token context limit. Dense pages (legal contracts, multi-column layouts) may contain 600–1200 OCR words. The current implementation truncates to the first 512 tokens (top-left reading order), risking missed answers in lower or right-column content. Sliding-window or hierarchical encoding strategies could address this.

C. Future Work

Lightweight distillation. LayoutLMv3-base (125M parameters) achieves 82.8% ANLS vs. UDOP's 84.7%—a 1.9% accuracy trade-off for a $6\times$ parameter reduction and corresponding inference speedup.

Retrieval-Augmented Generation. For multi-document binder analysis (e.g., 500-page due-diligence data rooms), a hybrid vector database (Pinecone, Weaviate, or pgvector) storing dense page embeddings would enable approximate nearest-neighbor retrieval to scale Smart-mode ranking to arbitrary corpus sizes.

OCR-free architectures. Donut [6] and Nougat [7] demonstrate end-to-end image-to-text performance without Tesseract, eliminating both the CPU-bound cold-start bottleneck and OCR error propagation. Integrating either as a drop-in replacement is the most impactful single improvement for DocChat.

VII. Conclusion

We presented DocChat, a fully operational Visual Document Question Answering system that successfully operationalizes Microsoft's UDOP transformer for real-time interactive document information extraction. Three contributions jointly address the deployment challenge: (1) UDOP's tri-modal spatial embeddings achieve 100% extraction precision on a custom three-document benchmark by correctly resolving layout-dependent field associations that defeat all text-only baselines; (2) a dual-level MD5-fingerprinted caching architecture reduces cold-start latency by $5.6\times$, enabling interactive query throughput; and (3) a TF-weighted Smart Page Ranking algorithm matches exhaustive All-Pages accuracy at roughly half the inference cost. Together, these contributions validate a general engineering principle: layout-aware multimodal pre-training, paired with intelligent amortization of preprocessing costs, can bridge the gap between computationally heavy academic vision-language models and the latency requirements of commercial real-time applications. Future directions include lightweight model distillation, RAG-based multi-document retrieval, and migration to fully OCR-free end-to-end architectures such as Donut or Nougat.

Acknowledgment

The authors thank the FAST School of Computing, NUCES Islamabad, for providing the computational resources and academic environment supporting this work, and the Computer Vision (Spring 2026) course instructors for their guidance throughout the project.

References

[1] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 2200–2209.

[2] J. Tang, C. Yang, S. Wang, H. Xu, B. Gao, C. Shi *et al.*, "Unifying vision, text, and layout for universal document processing," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19254–19264.

[3] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200.

[4] Y. Xu, Y. Xu, T. Lv, L. Cui, F. Wei, G. Wang *et al.*, "LayoutLMv2: Multi-modal pre-training for visually-rich document understanding," in *Proc. ACL-IJCNLP*, 2021, pp. 2579–2591.

[5] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. ACM Int. Conf. Multimedia (MM)*, 2022, pp. 4083–4091.

[6] G. Kim, T. Hong, M. Yim, J. Park, J. Yim, W. Hwang *et al.*, "OCR-free document understanding transformer," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 498–517.

[7] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," *arXiv preprint arXiv:2308.13418*, 2023.

[8] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, vol. 2, pp. 629–633.

[9] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP System Demonstrations*, 2020, pp. 38–45.

[10] M. Mathew, V. Bagal, R. Tito, D. Karatzas, E. Valveny, and C. V. Jawahar, "InfographicVQA," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2022, pp. 1697–1706.

[11] Microsoft Research, "UDOP: Unified document processing model card," Hugging Face Hub, 2022. [Online]. Available: https://huggingface.co/microsoft/udop-large-512-300k

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.